Test Generation System — Complete Technical Analysis

1. Architecture Pattern: Pure Rule-Based Deterministic Pipeline

Your system is **purely deterministic and rule-based**. No LLM is involved in test generation. The architecture follows a strict 4-module pipeline:

```
OpenAPI JSON/YAML
       ↓
[A] OpenApiParser         → ParsedApiSpec (endpoints, schemas, constraints, security)
       ↓
[B] ValidPayloadBuilder   → Schema-compliant payloads (seeded Random(42) for
reproducibility)
       ↓
[C] MutationEngine        → One mutation per test (missing field, wrong type, boundary
violation)
       ↓
[D] TestAssembler         → Final TestCaseDefinition objects with assertions,
severity, metadata
```

Module	Class	Responsibility
A	`OpenApiParser`	Parses OpenAPI 3.0 JSON/YAML into `ParsedApiSpec` (decoupled from library types)
B	`ValidPayloadBuilder`	Generates schema-compliant payloads using seeded `Random(42)` for reproducibility
C	`MutationEngine`	Applies exactly **one mutation per test** — isolates single failure reasons
D	`TestAssembler`	Orchestrates the pipeline, assembles final `TestCaseDefinition` objects with assertions

The SLM (Llama 3.1 / Phi-3.5) exists as a **separate assistive layer** — it's disabled by default and only extracts business rules from PR text. It never touches core generation.

---

2. How Test Categories Are Determined

The `TestAssembler.GenerateTestSuite()` method iterates every endpoint and applies category-specific generators **based on config flags and schema presence**:

Category	Condition to Generate	What Triggers It
Smoke (P0)	`_config.GenerateSmokeTests` (always true by default)	Every non-deprecated endpoint gets exactly 1 smoke test
Validation (P1)	`_config.GenerateValidationTests && endpoint.RequestBody != null`	Only endpoints with a request body (POST, PUT, PATCH)

Boundary (P2)	`_config.GenerateBoundaryTests && endpoint.RequestBody != null`	Only endpoints with request body + fields that have min/max/minLength/maxLength constraints
Negative (P1)	`_config.GenerateNegativeTests` (always true)	**Every endpoint, unconditionally ← This was a bug (now fixed)**
Contract (P1)	`_config.GenerateContractTests`	Only endpoints with 2xx response schemas defined
CRUD (P0)	`_config.GenerateCrudTests`	Resource groups that have at least POST + GET endpoints on the same path

---

3. What It Inspects From the OpenAPI Spec

✅ Fully Inspected

Element	Where Used	
HTTP methods	`TestAssembler` (smoke, CRUD detection)	De exp sta (PC DE etc
requestBody schema	`ValidPayloadBuilder.BuildPayload()`	Ge val fee `Mut`

response schema	`TestAssembler` (smoke + contract)	Ad
		`sch`
		`jsc`
		ass
		rec
		res

required fields	`MutationEngine.GenerateMissingRequiredFieldMutations()`	Re rec at exp 40
field types	`MutationEngine.GenerateWrongTypeMutations()`	Sw (st int exp 40
constraints (min/max/minLength/maxLength/enum)	`MutationEngine` boundary generators	Bo tes

		co ed
path parameters	`TestAssembler.BuildPathParams()`, `IsIdParameter()`	Ge inv exi ne
enum values	`MutationEngine.GenerateInvalidEnumMutations()`	Se en ex 40
additionalProperties	`TestAssembler` contract tests	As ext pro sch the

⚠️ Previously Broken (Now Fixed)

Element	Status
securitySchemes	`OpenApiParser` extracted operation-level security but **not global security**. `TestAssembler` **never checked it**. Both now fixed.

---

4. Why It Generated Auth Negative Tests for APIs With No Security

This was a bug. The `GenerateNegativeTests()` method looked like this:

```
private List<TestCaseDefinition> GenerateNegativeTests(ParsedEndpoint endpoint, ref
int counter)
{
    var tests = new List<TestCaseDefinition>();

    // No auth token → 401  ← UNCONDITIONAL — no security check!
    tests.Add(...);

    // Invalid auth token → 401  ← UNCONDITIONAL
    tests.Add(...);
    ...
}
```

There was no guard checking `endpoint.Security`. It unconditionally generated "No auth → 401" and "Invalid auth → 401" for every single endpoint, even APIs with no security defined.

---

5. Root Causes

Two separate issues caused this:

Issue A: Parser didn't capture global security

The `OpenApiParser.MapOperation()` extracted **operation-level** security:

```
if (operation.Security != null) { ... } // Only per-operation
```

But many APIs (like Petstore) define security **globally** in the spec root:

```
security:
  - api_key: []
  - petstore_auth: [write:pets, read:pets]
```

Global security was ignored → `endpoint.Security` was always empty.

Issue B: TestAssembler never checked security

Even if the parser had populated `endpoint.Security`, the `GenerateNegativeTests()` method never checked it. Auth tests were hardcoded to generate for all endpoints.

---

6. Fixes Applied

Fix 1: OpenApiParser now extracts global security

```
// Extract global security schemes
var globalSecurity = new List<string>();
if (doc.SecurityRequirements != null)
{
    foreach (var requirement in doc.SecurityRequirements)
    {
        foreach (var scheme in requirement.Keys)
        {
            globalSecurity.Add(scheme.Reference?.Id ?? scheme.Name ?? "unknown");
        }
    }
}

// If endpoint has no operation-level security, inherit global security
if (endpoint.Security.Count == 0 && globalSecurity.Count > 0)
{
    endpoint.Security.AddRange(globalSecurity);
}
```

Fix 2: TestAssembler guards auth tests behind security check

```
// Auth negative tests: ONLY generate if the endpoint has security requirements
defined
if (endpoint.Security.Count > 0)
{
    // No auth token → 401
```

```
    // Invalid auth token → 401
}
```

---

7. Impact on Test Counts

After fix, re-generating tests for an API with **no security defined** will produce:

- **Fewer negative tests** (no false 401 tests)
- **More accurate test suite** (only ID-related negatives: 400/404)

For APIs **with security** (like Petstore with `api_key` + `petstore_auth`):

- Auth tests still generated correctly
- No change in behavior

---

8. Module-by-Module Detail

Module A: OpenApiParser

- **Input:** Raw JSON/YAML string
- **Output:** `ParsedApiSpec` with fully resolved schemas, endpoints, parameters, constraints
- **Key:** Decouples from `Microsoft.OpenApi` library types for testability
- **Handles:** `$ref` resolution, enum extraction, constraint mapping, security inheritance

Module B: ValidPayloadBuilder

- **Input:** `ParsedSchema` from any endpoint's request body
- **Output:** Fully schema-compliant `Dictionary<string, object?>`
- **Key:** Seeded `Random(42)` ensures identical payloads every run (deterministic)
- **Priority:** Example value → Default value → First enum → Generated value
- **Format-aware:** Generates proper UUID, email, date-time, URI formats

Module C: MutationEngine

- **Input:** `ParsedSchema` + valid payload from Module B
- **Output:** `List<MutationResult>` — each with exactly ONE violated constraint
- **Mutations:** Missing required field, wrong type, invalid enum, additional property, string boundary (minLength-1, maxLength+1), numeric boundary (min-1, max+1), array boundary (minItems-1, maxItems+1)
- **Key principle:** One failure reason per test — never compound mutations

Module D: TestAssembler

- **Input:** Full `ParsedApiSpec` from Module A
- **Output:** `List<TestCaseDefinition>` — complete test suite
- **Orchestrates:** Calls B and C per endpoint, assigns severity, attaches assertions
- **CRUD detection:** Groups endpoints by resource path, identifies POST/GET/PUT/DELETE combinations, generates ordered sequences with dependencies

---

9. Strategic Positioning

Metric	Value

Deterministic coverage	~90% of all tests
AI-assisted	~10% (SLM rule extraction — disabled by default)
Reproducibility	100% (seeded random, same spec → identical tests)
Single point of failure per test	Enforced by MutationEngine
CI-blocking capability	Only deterministic tests; SLM tests require `needs_review: true`